

Arquitectura de Zuppeto

Document generat el 19 de juny de 2026 · Basat en `docs/ca/tecnic-ca.md` i l'estat actual del repositori

Zuppeto és una plataforma pet-friendly per descobrir llocs, estades i serveis que accepten mascotes. L'arquitectura actual combina un **frontend Angular 21** amb un **backend .NET** en capes (DDD), persistència a **PostgreSQL** i desplegament local via **Docker**.

1. Vista general

Usuari → Navegador → Angular Web (:4200) ↓ HTTP + JWT .NET API (:5211) ↓ Application →

Domain ↓ Infrastructure → PostgreSQL (:5433) ↓ Leaflet + OpenStreetMap (mapa, client)

El flux principal és: l'usuari interactua amb la web Angular → la web crida l'API REST → l'API delega en serveis d'aplicació → aquests usen el domini i repositoris → la infraestructura persisteix a PostgreSQL.

2. Estructura del repositori

Carpeta	Rol
<code>src/Web/</code>	Frontend Angular 21
<code>src/Backend/</code>	Backend .NET en 4 capes
<code>sql/init/</code>	Bootstrap auxiliar de BBDD (no l'esquema principal)
<code>docker/</code>	Dockerfiles de desenvolupament
<code>docker-compose.yml</code>	Stack local: db, api, web, rabbitmq
<code>e2e/</code>	Proves end-to-end amb Playwright

Carpeta	Rol
platform/	Paquets/miralls reutilitzables (p. ex. catàleg geogràfic)
docs/	Documentació funcional, tècnica i fases del projecte

3. Frontend (Angular 21)

Organitzat per **features**, amb tres capes transversals:

3.1 core/

Peces globals de l'aplicació:

- **Guards:** auth, guest, admin, permission
- **Interceptors:** autenticació JWT i errors HTTP
- **Layout:** capçalera, peu, notificacions d'error
- **Config:** URL de l'API

3.2 shared/

Components reutilitzables (place-card, city-combobox, favorite-toggle-button, etc.)

3.3 features/

Mòduls funcionals independents:

Feature	Responsabilitat
home	Landing, hero, ciutats trending
places	Cerca, filtres, mapa Leaflet, detall de lloc
favorites	Llista de favorits de l'usuari
auth	Login, perfil, callback OAuth

Feature	Responsabilitat
admin	Consola interna (usuaris, rols, permisos, menús, territori...)
help, contact, notifications	Pàgines de suport
Les rutes fan lazy loading de components i apliquen guards segons rol/permís. El mapa (place-map) és una capacitat transversal dins places, basada en Leaflet + OpenStreetMap .	

4. Backend (.NET) — arquitectura en capes (DDD)

El backend segueix **Domain-Driven Design** amb separació estricta:

Api (Minimal APIs + Swagger) ↓ Application (casos d'ús, DTOs, validadors) ↓ Domain

(agregats, value objects, regles) ↑ implementa contractes Infrastructure (EF,

repositoris, auth, integracions) ↓ PostgreSQL

4.1 Domain (src/Backend/Domain/)

Nucli de negoci, **sense dependències externes**:

- **Agregats:** Place, User, FavoriteList, PlaceReview
- **Value objects:** adreça, geolocalització, política pet, preu, rating, perfil, consentiment
- **Contractes:** IPlaceRepository, IUserRepository, etc.
- Base comuna: Entity, AggregateRoot, ValueObject, DomainRuleException

4.2 Application (src/Backend/Application/)

Orquestra els casos d'ús:

- Serveis: PlaceApplicationService, UserApplicationService, FavoriteListApplicationService, AdminApplicationService, AuthApplicationService...

- DTOs, validadors, commands/handlers
- **No coneix EF ni HTTP**

4.3 Infrastructure (src/Backend/Infrastructure/)

Implementacions concretes:

- **EF Core** amb models de persistència propis (*Record), separats del domini
- **Mappers manuals** per agregat (PlacePersistenceMapper, etc.)
- **Repositoris EF** que implementen els contractes del domini
- **Auth:** JWT, OAuth (Google, LinkedIn)
- **Integracions externes:** Google Places, GeoNames
- Migracions EF com a font de veritat de l'esquema

4.4 Api (src/Backend/Api/)

Capa HTTP fina:

- **Minimal APIs** agrupades per domini (PlaceEndpoints, UserEndpoints, AdminEndpoints...)
- **Swagger** per documentació i proves
- **JWT Bearer** i autorització per rol/permís
- No accedeix directament al DbContext

Flux d'una petició:

```
HTTP → Endpoint → Application Service → Repository (contracte)
      → EF Repository + Mapper → PostgreSQL
```

5. Persistència

- **PostgreSQL 17** via Docker (port extern 5433)
- L'esquema el governa **Entity Framework** (migracions a Infrastructure/Persistence/Migrations/)
- El domini **no es mapeja directament** a EF: hi ha models de persistència (*Record) i conversió explícita agregat ↔ record
- Caché de cerques de places (place_search_queries / place_search_query_results) amb TTL de 12 h

6. Infraestructura local (Docker)

Servei	Port	Funció
db	5433	PostgreSQL
api	5211	Backend .NET (build + migracions automàtiques)
web	4200	Angular dev server
rabbitmq	5672 / 15672	Missatgeria (preparada per a fases futures)

7. Seguretat i autorització (Fase IV)

Rol	Accés
VIEWER	Només lectura; sense favorits ni escriptura
USER	Producte funcional complet
DEVELOPER	Producte + documentació interna
ADMIN	Tot, inclòs manteniment d'usuaris/rols/permisos
• Login propi amb JWT + federació Google i LinkedIn • Catàleg de permisos per menu, page i action • El control s'aplica tant a Web (guards + permissionGuard) com a API (RequireAuthorization)	

8. Proves

- e2e/: Playwright — navegació, API, rols, seguretat, pantalles admin
- Resultats documentats a docs/probes-e2e-resultats/

9. Principis arquitectònics

1. **Frontend primer** (fases I-II amb dades fake), després backend real (Fase III tancada)

2. **Arquitectura per features** al frontend
3. **DDD + SOLID** al backend: domini net, inversió de dependències, repositoris com a ports
4. **Mapping manual** domini ↔ persistència (sense AutoMapper)
5. **Integracions externes només al backend** (Google Places, GeoNames)
6. **Internacionalització** prevista per al final (Fase V)

Referència principal: docs/ca/tecnic-ca.md · Estat de fases: docs/project-phases.md